Informationen zur Projektarbeit

A. Allgemeines

Teams

Die Projektarbeit wird in Teams von **maximal 4 Personen** erstellt.
Teams von mehr als 4 Personen oder Einzelpersonen sind *grundsätzlich* nicht möglich!
Die Teams (inkl. Projetthemen) müssen sich bis **zur nächsten Vorlesung in einer Woche** per E-Mail an **volker.stiehl@thi.de** verbindlich anmelden.

Bewertung

Wichtig – Vorgabe des Prüfungsamtes: Um eine individuelle Beurteilung zu ermöglichen, **muss die Arbeit aus individuell abgrenzbaren Anteilen bestehen**. D.h. **jedes Teammitglied** muss sich für einen bestimmten, substanziellen Code-Anteil **verantwortlich zeichnen**. Laut Prüfungsamt muss der *gesamte* Code unter den Team-Mitgliedern aufgeteilt werden.
Wer sich für welchen Code verantwortlich zeichnet, ist **als Kommentar direkt im Code zu vermerken**.

Während des Prüfungszeitraums stellen die Teams ihre Projektarbeit individuell vor. Überprüft wird u.a., ob die Anwendung fehlerfrei funktioniert und die Vorgaben erfüllt. Neben den erstellten Entwicklungsartefakten (z.B. BPMN-Modelle, Code) sind auch die in der Vorlesung behandelten Konzepte zu erklären! Projektarbeit und mündliche Abnahme gehen jeweils etwa zur Hälfte in die Gesamtbewertung ein.

Jedes einzelne Team-Mitglied muss dabei Fragen zu allen Bereichen des Codes bzw. der erstellten Modelle beantworten können, auch wenn sie diesen nicht selbst entwickelt haben! Eine Abnahme kann für ein individuelles Team-Mitglied auch dann als „nicht bestanden“ gewertet werden, obwohl eine lauffähige Prozessanwendung abgegeben wurde!

B. Aufgabe

Aufgabe ist es, eine **Prozessanwendung** bestehend aus einer **beliebigen Anzahl von Prozessen** zu implementieren**.** Dies kann sowohl eine betriebswirtschaftliche als auch eine eher technisch orientierte Anwendung (z.B. Koordinierung von Produktionsmaschinen) sein! Dabei müssen die im Folgenden aufgelisteten Vorgaben erfüllt werden:

Entwicklung (Mindestanforderungen):

- **Die Anwendung muss unter Verwendung von BPMN, Persistenz, Integration mit Datentransformation, Messaging, Eventing und Services (SOAP/REST) nach den Vorgaben des prozessgesteuerten Ansatzes umgesetzt werden**. Es ist wichtig, dass ein **kompletter vertikaler Durchstich** implementiert wird! Ziel ist die Implementierung einer Anwendung gemäß der **Referenzarchitektur für Anwendungssysteme** (wird in der Vorlesung behandelt).
  Zwingend zu berücksichtigende Punkte:
  - Versand einer asynchronen Nachricht aus dem Prozess heraus in eine JMS-Queue via Sende-Aufgabe
  - Mittels Apache Camel die Nachricht des Prozesses aus einer JMS-Queue lesen und an mehrere Systeme (in Form von Dateisystem oder DB-Tabelle) routen (Content-Based Routing – CBR). Dabei **bidirektionale** Transformation zwischen den Formaten XML und/oder JSON.
  - Sollte für das Routing die Best Practice aus der Vorlesung über Routing-Tabellen in BPMN gewählt werden, so kann CBR dennoch implementiert werden. Beispielsweise könnten bestimmte Inhalte aufgrund des Nachrichteninhalts gefiltert und nicht ausgeliefert werden.
  - Daten aus einem Dateisystem oder DB lesen und in JMS-Queue ablegen oder per Postman Daten in JMS-Queue schreiben (simuliert Versand einer Nachricht aus einem Backend-System)
  - Mittels Apache Camel Nachricht des „Backend-Systems“ entgegennehmen und an Prozess weiterleiten, der an einem nachrichtenbasierten Zwischenereignis wartet
- In der Vorlesung werden immer wieder „Best Practices“ zur Anwendung der verschiedenen Technologien präsentiert. Sie sind zwingend in der Anwendung anzuwenden!

- Die Verwendung bestimmter Frameworks, die die Implementierung erleichtern, ist zuvor mit dem Dozenten abzusprechen.
- Sie müssen in der Lage sein, sowohl die **Architektur** Ihrer Anwendung als auch die **Funktionsweise** Ihres Codes im Detail zu erklären.
- **Ausgefeilte Benutzeroberflächen werden NICHT verlangt**. Sparen Sie sich an dieser Stelle den Aufwand und konzentrieren Sie sich stattdessen auf eine funktionsfähige Gesamtarchitektur!
- **Während des Semesters wird in regelmäßigen Abständen der aktuelle Stand des Projektes in der Gruppe diskutiert**. Die Termine werden dafür vorab bekanntgegeben (in der Regel letzte November- und letzte Dezemberwoche).

Am **letzten Vorlesungstag vor Beginn des erweiterten Prüfungszeitraums** findet zudem ein **Pitch der Projekte** statt. Ziel ist es, die Lösungen an einen **imaginären Investor** zu verkaufen. Der Pitch besteht aus einer Kurzpräsentation und einer Demo der Anwendung. Bei der Präsentation sind neben den üblichen Inhalten (Was war das Problem? Wie wurde das Problem gelöst?) die folgenden beiden Punkte zu berücksichtigen:

- Warum sollte der Investor gerade in Eure Lösung investieren? Welche Lösungen gibt es bereits und was ist Euer Alleinstellungsmerkmal (USP)?
- Wie wollt Ihr mit der Lösung Geld verdienen? Wie groß ist der adressierbare Markt? Wie sieht also Euer Business Plan aus?

Orientiert Euch an dem für Projekt-Pitches bewährten Ablauf:

15 Minuten Pitch = 1 Problem, 1 Lösung, 1 Wow-Moment, 1 Geschäftsmodell, 1 klare Zukunft.

1. 1 Problem

Was enthalten sein sollte:

- Ein **konkretes, wiederkehrendes Problem** einer klar definierten Zielgruppe
- Ein **echtes Schmerzlevel** (Zeitverlust, Kosten, Risiko, Frust)
- Der heutige **Status quo** (Workarounds, Tools, manuelle Prozesse)
- Warum das Problem **relevant genug ist, um Geld dafür auszugeben**

Essenz:

„Hier ist ein echtes Problem, das häufig auftritt, messbaren Schaden verursacht und heute schlecht gelöst ist.“

---

2. 1 Lösung

Was enthalten sein sollte:

- Ein **klarer One-Liner**, der sofort verständlich ist
- Der **Kernnutzen**, nicht die Feature-Liste
- Für **wen** die Lösung gedacht ist (ideal: eine Hauptpersona)
- Warum eure Lösung **fundamental besser** ist als der Status quo

Essenz:

„So lösen wir genau dieses Problem einfacher, schneller oder besser als alles, was es heute gibt.“

---

3. 1 Wow-Moment

Was enthalten sein sollte:

- Ein **konkreter Use Case**, der live gezeigt wird (**Tipp**: Videoaufzeichnung des Use Cases parat haben, falls technische Probleme auftreten!)
- Ein Moment, der **Zeit spart, Komplexität entfernt oder etwas Neues ermöglicht**
- Ein klar messbares Ergebnis (z. B. Sekunden statt Stunden)
- Etwas, das man **nach dem Pitch weitererzählt**

Essenz:

„In diesem Moment versteht jeder intuitiv, warum das Produkt stark ist.“

---

4. 1 Geschäftsmodell

Was enthalten sein sollte:

- Wer **zahlt** und wofür
- Wie Geld verdient wird (Abo, Usage, Lizenz etc.)
- Grobe Preislogik (nicht jedes Detail)
- Warum das Modell **skalierbar und nachhaltig** ist

Essenz:

„Das ist kein nettes Tool – das ist ein funktionierendes, skalierbares Business.“

---

5. 1 klare Zukunft

Was enthalten sein sollte:

- Eine **ambitionierte, aber glaubwürdige Vision**
- Wie sich Produkt, Markt und Umsatz weiterentwickeln
- Welche Kategorie oder welchen Standard ihr prägen wollt
- Ein **klarer nächster Schritt** (Ask)

Essenz:

„Das hier ist erst der Anfang – und wir wissen genau, wohin die Reise geht."

Kompakt zusammengefasst (Pitch-kompatibel):

Problem: real, teuer, ungelöst
Lösung: einfach, fokussiert, besser
Wow: sofort verständlicher Nutzen
Business: skalierbar, klar monetarisiert
Zukunft: große Vision, klarer nächster Schritt

---

*Für **Code aus dem Internet** und für den **per künstlicher Intelligenz generierten Code** (z.B. ChatGPT) gelten folgende Regeln:*

- **Code aus dem Internet**: Beginn und Ende des übernommenen Codes müssen **eindeutig gekennzeichnet** sein. Außerdem müssen die URL der **Quelle** und (falls bekannt) der/die **Autor/in** angegeben werden. Es ist jedoch lediglich die Übernahme **einzelner Routinen** erlaubt. **Derart gekennzeichneter Code muss während der Abnahme nicht erklärt werden**!
- **Per künstlicher Intelligenz generierter Code**: Auch hier müssen Beginn und Ende des übernommenen Codes **eindeutig gekennzeichnet** werden. Außerdem ist **die zur Codegenerierung genutzte künstliche Intelligenz zu nennen** (z.B. ChatGPT). Ist der überwiegende Teil des Codes von der künstlichen Intelligenz erzeugt worden, so dreht sich die Beschriftung um: Als Hauptautor ist die genutzte künstliche Intelligenz zu nennen. Jetzt müssen **Beginn und Ende des selbst entwickelten Codes eindeutig gekennzeichnet** werden. **Im Unterschied zu Code aus dem Internet muss der per künstlicher Intelligenz erzeugte Code während der Abnahme erklärt werden können**!

- Bei Zweifeln über die Zulässigkeit von übernommenem Code bitte frühzeitig nachfragen und abklären! Sie können jederzeit auch individuelle Besprechungstermine mit mir vereinbaren.
- Der Sourcecode zum Projekt und ein Datenbankabzug müssen **am letzten Tag vor Beginn des erweiterten Prüfungszeitraumes** per E-Mail an `volker.stiehl@thi.de` geschickt werden, damit ein Plagiatscheck durchgeführt werden kann. Wird dabei ungekennzeichneter Code aus dem Internet entdeckt, dann wird dies als Täuschungsversuch bei der Prüfungskommission angezeigt, die dies in der Regel mit der Note 5,0 bewertet.
- Nach Abgabe des Sourcecodes dürfen **keinerlei Veränderungen mehr am Code durchgeführt werden**. Sollten Sie bis zum Abnahmetermin Fehler entdecken, so dokumentieren Sie diese Fehler in Form einer Fehlerbeschreibung in einem Textdokument. Dieses Dokument geben sie dem Prüfer am Termin ihrer Abnahme gleich zu Beginn des Termins ab. Sollte während der Abnahme der Fehler auftreten und er ist entsprechend dokumentiert, gilt dieser Fehler als „nicht vorhanden".
- Weicht der während der Abnahme durchlaufene Code vom abgegebenen Code ab, so gilt dies als Täuschungsversuch und wird mit der Note 5,0 bewertet.

Programmierumgebungen / Tools / Hilfsmittel

Als Entwicklungsumgebung wird Eclipse IDE für Jakarta EE Entwickler und als Datenbank MySQL empfohlen.

Folgende Frameworks und Engines kommen zum Einsatz:

- Java Development Kit (JDK) 17
- Quarkus als Framework zur Erstellung von Microservices
- CIB seven BPMN-basierte Process Engine
- Apache ActiveMQ (Classic) Messaging Framework
- Apache Camel Integration Framework
- Esper ESP Engine

C. Mögliche Prozesse

- **Wissensautomatisierungsprozesse**
- Genehmigungsprozess
- Antragsprozess
- Registrierungsprozess
- Maschinensteuerungsprozess
- Supportprozess
- Produktentwicklungsprozess
- Produktionsprozess
- Serviceprozess (z.B. Einsatz eines Service-Mitarbeiters im Außendienst)
- Beschwerdemanagement
- Schadensabwicklungsprozess
- Inventurprozess
- Einstellungsprozess
- Angebotsprozess
- Wareneingangsprozess
- Bestellprozess
- Rechnungseingangsprozess
- Einkaufs-/Beschaffungsprozess
- Vermietungsprozess
- Dienstreiseprozess
- Einsatzplanungsprozess
- Flugtaxibestell- und -abwicklungsprozesse
- Corona-Testprozesse
- Eventorganisation und -abwicklung zu Pandemiezeiten
- Weltraumtouristik: Raketen-Landeverwaltung
- E-Scooter-Verleih
- Raumbuchungssystem
- Whistleblower-Plattform
- Zufahrtsbeschränkung für Innenstädte auf Basis eines automatisierten Schrankensystems
- Abläufe in Krankenhäusern
- …